ReservaYa: sistema web de reservas en línea para restaurantes pequeños y medianos

Actividad 2. Documento de formulación del proyecto de software

Saleth Tatiana Ibarra Martínez

ID: 100171247

Kevin Smith Correal Chocontá

ID: 100173678

José Leonardo Guzmán Tobón

ID: 100169251

Facultad de Ingeniería, Ingeniería de Software

Corporación Universitaria Iberoamericana

Proyecto de Software

Docente: Ana Mosquera

Colombia

28 de septiembre de 2026

Tabla de contenido

Introducción 4
1. Descripción del problema 5
Situación actual 5
Necesidad del sector productivo 6
Posibles soluciones 6
2. Alcance del proyecto 7
Qué incluye y qué no incluye 8
Restricciones 8
Criterios de aceptación del proyecto 9
3. Estructura de desglose del trabajo (EDT) 10
4. Objetivos 13
Objetivo general 13
Objetivos específicos 13
5. Justificación 14
Soluciones al problema 14
Respuesta a los stakeholders 15
6. Matriz de riesgos 15
7. Cronograma 17
8. Presupuesto 20
9. Repositorio del proyecto 22
Conclusiones 24
Referencias 25

Tabla de ilustraciones

Tabla 1 Alternativas de solución para la gestión de reservas 6
Figura 1 Arquitectura propuesta para ReservaYa versión 1.0 7
Tabla 2 Brecha entre el prototipo actual (v0.1) y la versión 1.0 8
Tabla 3 Delimitación del alcance 8
Tabla 4 Restricciones del proyecto 9
Tabla 5 Criterios de aceptación del proyecto 9
Figura 2 Estructura de desglose del trabajo de ReservaYa versión 1.0 10
Tabla 6 Diccionario de la EDT 11
Tabla 7 Relación entre objetivos, EDT e indicadores 14
Tabla 8 Alcance de la justificación en el corto, mediano y largo plazo 14
Tabla 9 Correspondencia entre problemas y soluciones 15
Tabla 10 Stakeholders del proyecto y respuesta a sus intereses 15
Tabla 11 Escalas de probabilidad e impacto 16
Figura 3 Mapa de calor de los riesgos del proyecto 16
Tabla 12 Matriz de riesgos del proyecto 17
Tabla 13 Plan de sprints e hitos del proyecto 18
Tabla 14 Cronograma detallado de los paquetes de trabajo 18
Figura 4 Diagrama de Gantt del proyecto 19
Tabla 15 Presupuesto del proyecto en pesos colombianos 21
Tabla 16 Resumen del presupuesto por rubro 22
Tabla 17 Información del repositorio 22

Introducción

En Colombia, operan alrededor de 150.000 restaurantes en todo el territorio; sin embargo, el sector atraviesa un momento difícil, cerca de 7.000 establecimientos cerraron sus puertas en 2024 y más de 2.000 en 2025, esto debido al alza de los insumos, de los servicios públicos y de los costos laborales (Múnera Zambrano, 2026). En este contexto, cada mesa que queda vacía por una reserva mal efectuada, un cliente que no llega o una llamada que nadie contestó, es dinero que un restaurante de pequeño o mediano tamaño no puede darse el lujo de perder. De igual manera, la mayoría sigue recibiendo sus reservas por teléfono, WhatsApp o incluso, en un cuaderno, esto genera problemas como mala reputación y reseñas negativas, cancelaciones a última hora, clientes esperando más de lo prometido y saturación de la cocina y sala por la mala gestión de las reservas (La casa del camarero, 2026).

En respuesta a esta situación, nació **ReservaYa**. En el primer semestre de 2026 el equipo desarrolló un prototipo web en Angular que ya permite buscar restaurantes, ver su detalle, diligenciar una reserva y obtener un código de confirmación, además de un panel administrativo de demostración. Sin embargo, este prototipo solo funciona con datos simulados; no guarda las reservas, no controla la disponibilidad real de las mesas y no avisa a nadie. La idea principal de este proyecto es transformarlo en un producto que un restaurante pueda usar para su operación diaria. Todo esto dentro de la versión 1.0 de ReservaYa que se planea desarrollar.

Este documento corresponde a la etapa de planeación, que según Pressman (2021) debe establecer el alcance, las tareas, los riesgos, los recursos y el calendario antes de comprometer esfuerzo en la construcción. Como reflexión inicial, reconocemos que un prototipo que se ve bien no equivale a un producto que funcione. La parte visible de **ReservaYa** ya existe; lo que falta es lo que no se ve: persistencia, control de concurrencia, seguridad y protección de datos.

1. Descripción del problema

Situación actual

Los restaurantes pequeños y medianos, suelen recibir las reservas por tres canales al mismo tiempo: llamadas al teléfono fijo o celular, mensajes de WhatsApp y clientes que se acercan en persona. La información queda repartida entre el chat de WhatsApp, un cuaderno en la caja y la memoria de quien atendió y agendó la reserva. De esta forma de trabajar se derivan cinco problemas concretos:

1. **Doble asignación de mesas:** Dos personas del equipo prometen la misma mesa a la misma hora porque no consultan una fuente única.
2. **Reservas perdidas:** Un mensaje llega en plena hora pico, nadie lo responde a tiempo y el cliente termina buscando otro lugar.
3. **Ausentismo sin control (no-show):** Nadie recuerda la reserva al cliente, de modo que la mesa se guarda para alguien que no llega. En España, donde plataformas como *TheFork* miden este fenómeno, el ausentismo promedio fue de 3,3 % en 2025 y ha bajado gracias a medidas como la reconfirmación automática (Plaza, 2026).
4. **Sin comprobante para el cliente:** El comensal no recibe un código ni una confirmación verificable, y si hay un malentendido no hay cómo demostrar lo acordado.
5. **Sin datos para decidir:** El administrador no sabe qué días y horas tienen más demanda, cuántas reservas se cancelan o cuántos clientes no llegan, información que necesitaría para ajustar personal y compras.

Estos problemas pesan más en un sector con márgenes estrechos. Acodrés reporta que muchos negocios han reducido la nómina y que la informalidad crece porque los costos no dan espera (Múnera Zambrano, 2026). Para un restaurante pequeño o mediano en esa situación,

llenar las mesas que ya tiene es más urgente que abrir nuevas. Cabe aclarar que los problemas específicos son supuestos mientras no se realicen las entrevistas.

Necesidad del sector productivo

El sector productivo específico es el de restaurantes pequeños y medianos. Su necesidad es contar con un canal de reservas que funcione solo, sin que alguien tenga que estar pendiente del teléfono, que evite asignar dos veces la misma mesa, que recuerde la cita al cliente y que entregue datos básicos de ocupación, todo a un costo que un negocio pequeño pueda asumir. El equipo validará estos supuestos con entrevistas a administradores y comensales de Medellín durante el sprint 0 (paquete 2.1 de la EDT), siguiendo la recomendación de Sommerville (2005) de obtener los requerimientos directamente de los interesados antes de diseñar.

Posibles soluciones

La Tabla 1 compara las alternativas que tiene hoy un restaurante pequeño o mediano para organizar sus reservas.

Tabla 1

Tabla 1 Alternativas de solución para la gestión de reservas

Alternativa	Ventajas	Desventajas
Seguir con teléfono, WhatsApp y cuaderno	Costo cero; el personal ya sabe usarlo	Mantiene los cinco problemas descritos; depende de una sola persona
Hoja de cálculo compartida	Bajo costo; centraliza la información	No evita cruces en tiempo real, no confirma ni recuerda, el cliente no la ve
Plataforma internacional de reservas	Producto maduro y con muchas funciones	Pensada para mercados grandes; cobra suscripción o comisión en dólares; poca adaptación local
ReservaYa v1.0 (propuesta)	Disponibilidad real, código de confirmación, recordatorios y estadísticas; costo bajo; adaptada al contexto local	Requiere construir backend, lograr adopción y sostener la operación

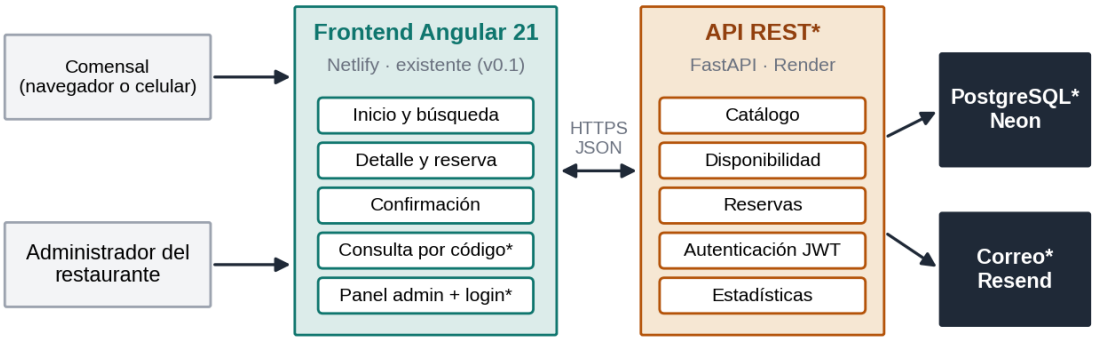
Proponemos la cuarta alternativa ya que ataca los cinco problemas anteriormente descritos y parte de un prototipo que ya valida la experiencia del comensal. El problema queda formulado así: **¿cómo podemos lograr que un restaurante pequeño o mediano de Medellín reciba y gestione sus reservas en línea sin asignar dos veces una mesa, reduciendo el ausentismo y obteniendo datos de ocupación, con un costo de operación mínimo?**

2. Alcance del proyecto

El proyecto entregará la versión 1.0 de ReservaYa: el frontend existente pero conectado a una API REST nueva, con base de datos, correo y autenticación para el administrador (Figura 1). Siguiendo el principio de manufactura esbelta de eliminar lo que no agrega valor al cliente, se incluye solo lo necesario para que un restaurante reciba reservas reales, y lo demás queda para versiones posteriores.

Figura 1

Figura 1 Arquitectura propuesta para ReservaYa versión 1.0



** Componente nuevo en la versión 1.0. El frontend actual funciona con datos simulados.*

Nota. Elaboración propia. Los componentes marcados con asterisco se construyen en este proyecto; el frontend ya existe en el repositorio.

La Tabla 2 muestra la brecha entre lo que hace hoy el prototipo y lo que hará la versión 1.0, con base en la revisión del código del repositorio.

Tabla 2

Tabla 2 Brecha entre el prototipo actual (v0.1) y la versión 1.0

Funcionalidad	Estado actual (v0.1)	Alcance propuesto (v1.0)
Restaurantes	Datos simulados	Gestión mediante backend
Reservas	No persistentes	Persistencia en BD
Código de confirmación	Generado localmente	Código almacenado y consultable
Notificaciones	No existen	Correo de confirmación, recordatorio 24 h antes y aviso de cancelación
Panel administrativo	Demostrativo	Gestión autenticada
Estadísticas	Datos de prueba	Datos reales

Qué incluye y qué no incluye

Tabla 3

Tabla 3 Delimitación del alcance

Incluido en la versión 1.0	Fuera del alcance (versiones posteriores)
API REST con base de datos PostgreSQL	Pagos en línea, anticipos o garantía con tarjeta
Disponibilidad por mesa y franja horaria, con bloqueo de fechas	Aplicación móvil nativa (Android o iOS)
Reserva con código único, consulta y cancelación por el cliente	Integración con sistemas POS o de facturación
Correos de confirmación, recordatorio y cancelación	Notificaciones por WhatsApp o SMS
Panel administrativo con autenticación y estadísticas básicas	Cuentas de comensal, programa de fidelización y reseñas

Restricciones

Las restricciones son condiciones que el proyecto no puede cambiar y que limitan cómo se ejecuta (Tabla 4).

Tabla 4

Tabla 4 Restricciones del proyecto

Tipo	Restricción	Efecto en el proyecto
Tiempo	Tiempo académico limitado	Priorización de funcionalidades
Equipo	Equipo de tres integrantes	Distribución de responsabilidades
Costo	Sin financiación externa y con recursos económicos limitados	Uso de herramientas gratuitas
Tecnología	Se conserva el frontend en Angular 21, Tailwind CSS y Vitest	El backend debe exponer una API REST compatible con el frontend actual
Alcance	Proyecto académico	Alcance limitado
Infraestructura	Infraestructura disponible	Condiciona despliegue

Criterios de aceptación del proyecto

El proyecto se dará por aceptado cuando se cumplan los criterios de la Tabla 5, cada uno con una forma concreta de comprobarlo.

Tabla 5

Tabla 5 Criterios de aceptación del proyecto

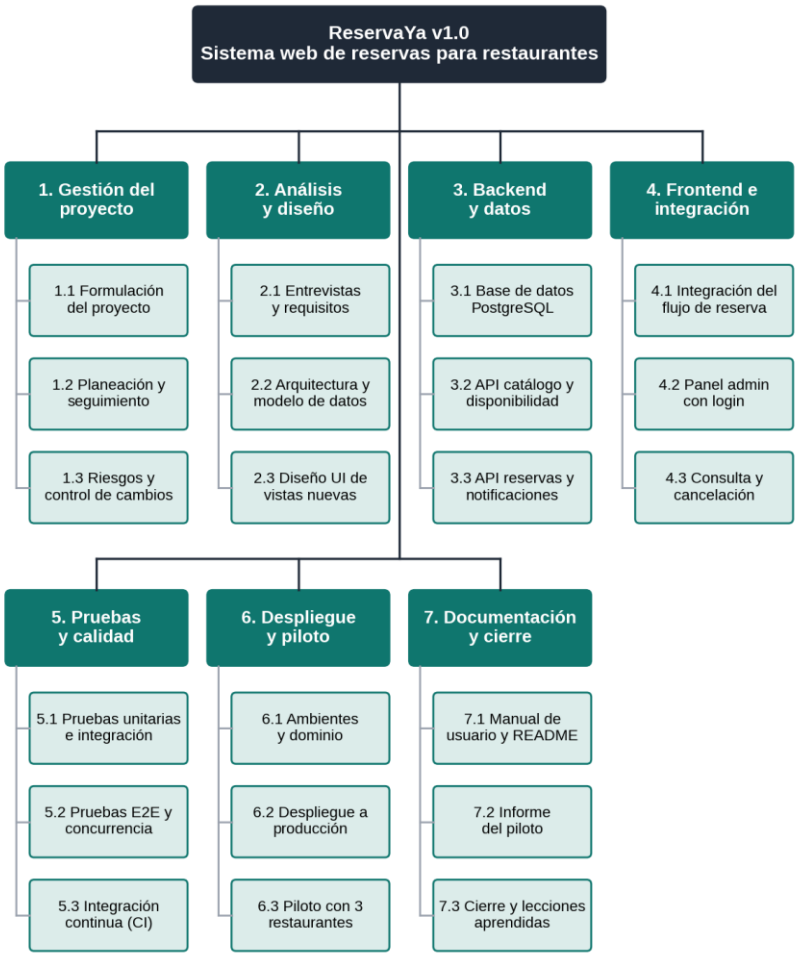
ID	Criterio	Cómo se comprueba
CA01	El usuario puede completar el flujo de reserva.	Una prueba de extremo a extremo recorre búsqueda, detalle, reserva y confirmación sin errores.
CA02	Disponibilidad y prevención de reservas duplicadas.	Se envían 20 solicitudes simultáneas a la misma mesa y franja, y solo una queda registrada.
CA03	Confirmaciones, recordatorios y cancelaciones.	Los correos de confirmación, recordatorio (24 h antes) y cancelación llegan durante el piloto.
CA04	El administrador puede gestionar reservas.	El administrador inicia sesión y crea, edita y cancela reservas, mesas y horarios.
CA05	Las funcionalidades principales cumplen las pruebas definidas.	Las pruebas pasan en integración continua con cobertura mínima de 70 % en la lógica de negocio.
CA06	La aplicación funciona correctamente en dispositivos de diferentes tamaños.	Las vistas se revisan en celular, tableta y computador sin que nada se descuadre.
CA07	Tiempos de respuesta y pruebas del sistema.	Las consultas principales responden en menos de 2 segundos en producción.
CA08	Seguridad al consultar o cancelar una reserva mediante código.	Sin el código correcto no se puede ver ni cancelar una reserva, y los códigos no son consecutivos ni fáciles de adivinar.

3. Estructura de desglose del trabajo (EDT)

La EDT descompone jerárquicamente todo el trabajo del proyecto en partes más pequeñas y manejables, de modo que la suma de los paquetes de trabajo equivale al 100 % del alcance (Project Management Institute [PMI], 2021). En ReservaYa la organizamos por entregables y no por actividades, porque lo que le importa al restaurante, y lo que se revisa al final de cada sprint, es lo que se entrega. Tiene tres niveles: el producto, siete entregables principales y 21 paquetes de trabajo (Figura 2). La Tabla 6 funciona como diccionario de la EDT: describe el resultado de cada paquete, quién responde por él y en qué sprint se ejecuta.

Figura 2

Figura 2 Estructura de desglose del trabajo de ReservaYa versión 1.0



Nota. Elaboración propia. La EDT está orientada a entregables; el primer dígito identifica el entregable principal.

Los siete entregables recorren el proyecto de principio a fin. La gestión del proyecto (1) reúne lo que permite planear y controlar el trabajo; el análisis y diseño (2) recoge lo que hay que saber y decidir antes de programar; el backend y datos (3) y el frontend e integración (4) son el producto en sí; las pruebas y calidad (5) aseguran que lo construido funcione como se prometió; el despliegue y piloto (6) lleva la aplicación a restaurantes reales, y la documentación y cierre (7) deja el proyecto listo para continuar con una segunda versión.

Cada paquete de trabajo se convierte en una o varias historias de usuario en el tablero de GitHub Projects y conserva su código, por ejemplo «3.3 API de reservas», de modo que el avance del tablero se puede leer directamente contra la EDT. Si durante el semestre aparece trabajo que no está en ningún paquete, no se agrega sin más: pasa por el control de cambios del paquete 1.3 y, si el Product Owner lo aprueba, entra a la EDT desplazando algo de igual tamaño. Así evitamos que el alcance crezca sin que nadie lo note, que es justamente el riesgo R03.

Tabla 6

Tabla 6 Diccionario de la EDT

Código	Paquete de trabajo	Resultado esperado	Responsable	Sprint
1.1	Formulación del proyecto	Este documento aprobado y publicado en el repositorio	Equipo	0
1.2	Planeación y seguimiento	Backlog priorizado, tablero actualizado, actas de revisión y retrospectiva	Kevin	0 a 4
1.3	Riesgos y control de cambios	Matriz de riesgos revisada cada sprint y registro de cambios aprobados	Kevin	0 a 4
2.1	Entrevistas y requisitos	5 entrevistas a administradores y 5 a comensales; historias de usuario con criterios	Saleth	0
2.2	Arquitectura y modelo de datos	Diagrama de arquitectura, modelo entidad-relación y contrato de la API (OpenAPI)	José Leonardo	0 y 1

Código	Paquete de trabajo	Resultado esperado	Responsable	Sprint
2.3	Diseño UI de vistas nuevas	Diseños de login, consulta por código y gestión de mesas	Saleth	1
3.1	Base de datos PostgreSQL	Tablas, restricciones de unicidad y migraciones	José Leonardo	1
3.2	API de catálogo y disponibilidad	Servicios de restaurantes, filtros y franjas libres	José Leonardo	1 y 2
3.3	API de reservas y notificaciones	Creación transaccional de reservas, código único y correos	José Leonardo	2
4.1	Integración del flujo de reserva	Búsqueda, detalle, reserva y confirmación consumiendo la API	Kevin	2
4.2	Panel admin con login	Autenticación, gestión de reservas, mesas, horarios y estadísticas	Kevin	3
4.3	Consulta y cancelación	Vista para consultar y cancelar con el código de reserva	Saleth	3
5.1	Pruebas unitarias e integración	Suites de frontend y backend con cobertura de 70 % en la lógica de negocio	Equipo	1 a 4
5.2	Pruebas E2E y concurrencia	Flujos completos automatizados y prueba de 20 solicitudes simultáneas	Saleth	3 y 4
5.3	Integración continua (CI)	GitHub Actions que ejecuta pruebas en cada pull request	José Leonardo	1
6.1	Ambientes y dominio	Ambientes de pruebas y producción, dominio y correo configurados	José Leonardo	1
6.2	Despliegue a producción	Versión 1.0 publicada y accesible	José Leonardo	3
6.3	Piloto con 3 restaurantes	Restaurantes capacitados y operando con reservas reales	Saleth	4
7.1	Manual de usuario y README	Manual del administrador y README técnico actualizados	Kevin	4
7.2	Informe del piloto	Métricas del piloto frente a los criterios de aceptación	Saleth	4
7.3	Cierre y lecciones aprendidas	Acta de cierre, retrospectiva final y backlog de la versión 2	Equipo	4

Nota. Elaboración propia. Un paquete se considera terminado cuando cumple sus criterios de aceptación, tiene pruebas, pasó revisión de código y el Product Owner lo acepta.

4. Objetivos

Objetivo general

Desarrollar la versión 1.0 de ReservaYa, un sistema web que permita a los comensales reservar mesa en restaurantes pequeños y medianos con disponibilidad real y confirmación inmediata, y a los administradores gestionar reservas, mesas y horarios con datos de ocupación, validándola en una prueba piloto con tres restaurantes de Medellín, entre septiembre y noviembre de 2026.

Objetivos específicos

1. **Analizar** el proceso actual de reservas en restaurantes de Medellín mediante entrevistas a administradores y comensales, para priorizar los requisitos de la versión 1.0.
2. **Diseñar** la arquitectura cliente-servidor y el modelo de datos relacional que permitan controlar la disponibilidad por mesa y franja horaria sin reservas duplicadas.
3. **Construir** una API REST con persistencia en PostgreSQL y notificaciones por correo, e integrarla al frontend Angular existente en reemplazo de los datos simulados.
4. **Implementar** el panel administrativo con autenticación, gestión de reservas, mesas y horarios, y estadísticas de ocupación y ausentismo.
5. **Verificar** la calidad del sistema mediante pruebas automatizadas unitarias, de integración, de extremo a extremo y de concurrencia, ejecutadas en integración continua.
6. **Desplegar** la aplicación en producción y **evaluar** su funcionamiento en una prueba piloto con tres restaurantes, comparando los resultados con los criterios de aceptación.

La Tabla 7 relaciona cada objetivo específico con los paquetes de la EDT que lo cumplen y con el indicador que demuestra su logro.

Tabla 7

Tabla 7 Relación entre objetivos, EDT e indicadores

Objetivo	Paquetes EDT	Indicador de logro
1. Analizar	2.1	10 entrevistas realizadas y backlog priorizado con MoSCoW
2. Diseñar	2.2, 2.3, 3.1	Modelo de datos con restricción de unicidad mesa-franja aprobado en revisión
3. Construir	3.2, 3.3, 4.1	Criterios CA01, CA02 y CA03 cumplidos
4. Implementar	4.2, 4.3	Criterios CA04 y CA08 cumplidos
5. Verificar	5.1, 5.2, 5.3	Criterios CA05 y CA07 cumplidos
6. Desplegar y evaluar	6.1, 6.2, 6.3, 7.2	Criterio CA06 cumplido, versión 1.0 en producción e informe del piloto entregado

5. Justificación

ReservaYa se justifica porque resuelve un problema operativo concreto de un sector que hoy no tiene margen para desperdiciar mesas, y porque lo hace con una inversión mínima al aprovechar un prototipo ya construido y servicios en la nube con planes gratuitos. Su valor se proyecta en tres horizontes (Tabla 8).

Tabla 8

Tabla 8 Alcance de la justificación en el corto, mediano y largo plazo

Horizonte	Resultado esperado
Corto plazo	Tener la versión 1.0 en producción y validada con tres restaurantes en la prueba piloto.
Mediano plazo	Agregar lo que quedó por fuera de la versión 1.0: anticipos en línea, avisos por WhatsApp y cuentas de comensal.
Largo plazo	Ofrecer ReservaYa a más restaurantes de la región e integrarla con sistemas POS y de facturación.

Soluciones al problema

Cada uno de los problemas identificados en la sección 1 tiene una respuesta concreta dentro del alcance (Tabla 9).

Tabla 9

Tabla 9 Correspondencia entre problemas y soluciones

Problema	Solución en ReservaYa v1.0
Doble asignación de mesas	Una sola fuente de verdad en base de datos, con restricción de unicidad por mesa y franja y creación transaccional de la reserva
Reservas perdidas	Reserva en línea disponible las 24 horas, sin depender de que alguien conteste
Ausentismo sin control	Recordatorio automático 24 horas antes, cancelación en línea que libera la mesa y registro de asistencia
Sin comprobante para el cliente	Código único con formato RY-AAAAMMDD-XXXX, enviado por correo y consultable en la web
Sin datos para decidir	Panel con ocupación semanal, reservas por estado y tasa de ausentismo

Respuesta a los stakeholders

La Tabla 10 identifica a los interesados del proyecto, lo que esperan y cómo responde ReservaYa a cada uno.

Tabla 10

Tabla 10 Stakeholders del proyecto y respuesta a sus intereses

Stakeholder	Interés principal	Respuesta del proyecto
Comensales	Reservar rápido, con certeza de que la mesa estará lista	Disponibilidad real, código de confirmación y recordatorio
Propietarios y administradores de restaurantes	Llenar mesas, evitar errores y conocer su ocupación	Panel de gestión, bloqueo de fechas y estadísticas
Equipo de desarrollo	Aplicar lo aprendido en un producto real y ampliar su portafolio	Roles Scrum definidos, repositorio público y piloto con usuarios reales
Docente y Corporación Universitaria Iberoamericana	Evidencia de gestión de proyectos de software	Documentación completa en APA 7 y seguimiento en GitHub

6. Matriz de riesgos

La gestión de riesgos se planteó de forma proactiva: identificar lo que puede salir mal antes de que ocurra, estimar su probabilidad y su impacto, y definir de antemano cómo reducirlo, vigilarlo y manejarlo (Pressman, 2021). Cada riesgo se calificó de 1 a 5 en probabilidad y en impacto según la Tabla 11; el nivel es el producto de ambos valores.

Tabla 11

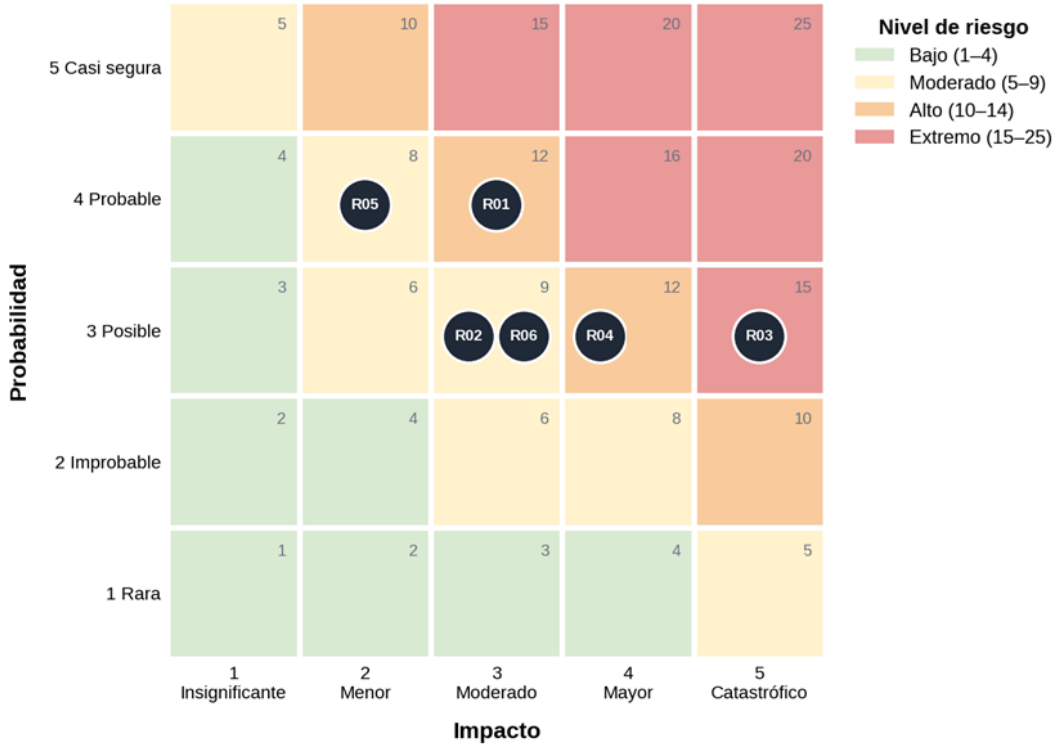
Tabla 11 Escalas de probabilidad e impacto

Valor	Probabilidad	Impacto en el proyecto
1	Rara: menos de 10 %	Insignificante: se absorbe sin cambiar el plan
2	Improbable: 10 % a 30 %	Menor: retraso de menos de 2 días
3	Posible: 30 % a 50 %	Moderado: retraso de hasta una semana o recorte de una función secundaria
4	Probable: 50 % a 80 %	Mayor: afecta un hito o una función principal
5	Casi segura: más de 80 %	Catastrófico: impide cumplir los criterios de aceptación o genera un incidente legal

La Figura 3 ubica los seis riesgos identificados en el mapa de calor, y la Tabla 12 detalla cada uno con su estrategia de respuesta y su responsable.

Figura 3

Figura 3 Mapa de calor de los riesgos del proyecto



Nota. Elaboración propia. El número en la esquina de cada celda es el nivel de riesgo (probabilidad × impacto).

Tabla 12

Tabla 12 Matriz de riesgos del proyecto

ID	Riesgo	Categoría	P	I	Nivel	Estrategia y respuesta	Resp.
R01	Poca disponibilidad del equipo por carga académica y laboral, que retrasa las entregas	Cronograma	4	3	12 Alto	Mitigar: capacidad planeada en 12 h por persona, holgura en el sprint 4 y reunión diaria de 15 minutos	Kevin
R02	Reservas duplicadas por solicitudes simultáneas a la misma mesa y franja	Técnico	3	3	9 Moderado	Evitar: restricción de unicidad en la base de datos, transacción al crear la reserva y prueba de concurrencia	José Leonardo
R03	Cambios de alcance durante el semestre (*scope creep*)	Alcance	3	5	15 Extremo	Mitigar: backlog priorizado con MoSCoW; todo cambio pasa por el Product Owner y desplaza algo de igual tamaño	Saleth
R04	Pérdida del código fuente o de la base de datos	Técnico	3	4	12 Alto	Mitigar: GitHub con rama principal protegida, respaldo semanal exportado y migraciones versionadas	José Leonardo
R05	Ausencia temporal o retiro de un integrante	Equipo	4	2	8 Moderado	Mitigar: revisión de código cruzada, documentación en el README y tareas que conozcan al menos dos personas	Kevin
R06	Conflictos de dependencias de Angular 21 al instalar o actualizar	Técnico	3	3	9 Moderado	Aceptar: versiones fijadas en package-lock y compilación en CI con entorno limpio	Kevin

Nota. P = probabilidad; I = impacto. Elaboración propia. La matriz se revisa en cada retrospectiva y el Scrum Master actualiza las calificaciones.

7. Cronograma

El proyecto se organiza con Scrum en cinco sprints de dos semanas, del 21 de septiembre al 27 de noviembre de 2026, lo que suma 47 días hábiles una vez descontados los festivos. Cada sprint cierra con una revisión, donde se muestra el incremento funcionando, y con una

retrospectiva. La Tabla 13 resume el objetivo y el hito de cada sprint, la Tabla 14 baja ese plan al nivel de los 21 paquetes de trabajo de la EDT y la Figura 4 presenta el diagrama de Gantt.

Tabla 13

Tabla 13 Plan de sprints e hitos del proyecto

Sprint	Fechas (2026)	Objetivo del sprint	Hito al cierre
0	21 sept. a 2 oct.	Formular el proyecto, entrevistar a restaurantes y comensales, y definir arquitectura	H1. Formulación entregada (2 oct.)
1	5 a 16 oct.	Crear base de datos, API de catálogo, integración continua y ambientes	H2. API y base de datos en ambiente de pruebas (16 oct.)
2	19 a 30 oct.	Reservar con disponibilidad real y enviar el correo de confirmación	H3. Flujo de reserva con datos reales (30 oct.)
3	3 a 13 nov.	Panel administrativo con login, consulta por código y despliegue	H4. Versión 1.0 en producción (13 nov.)
4	17 a 27 nov.	Piloto con 3 restaurantes, corrección de defectos, informe y cierre	H5. Cierre del proyecto (27 nov.)

Nota. Elaboración propia. Los sprints 1, 3 y 4 tienen nueve días hábiles por los festivos del 12 de octubre, 2 de noviembre y 16 de noviembre.

La Tabla 14 muestra cuándo empieza y termina cada paquete de trabajo. Las duraciones están en días hábiles y la columna «Depende de» indica qué paquete debe estar terminado, o por lo menos avanzado, para poder empezar. Los paquetes marcados con asterisco forman la ruta crítica.

Tabla 14

Tabla 14 Cronograma detallado de los paquetes de trabajo

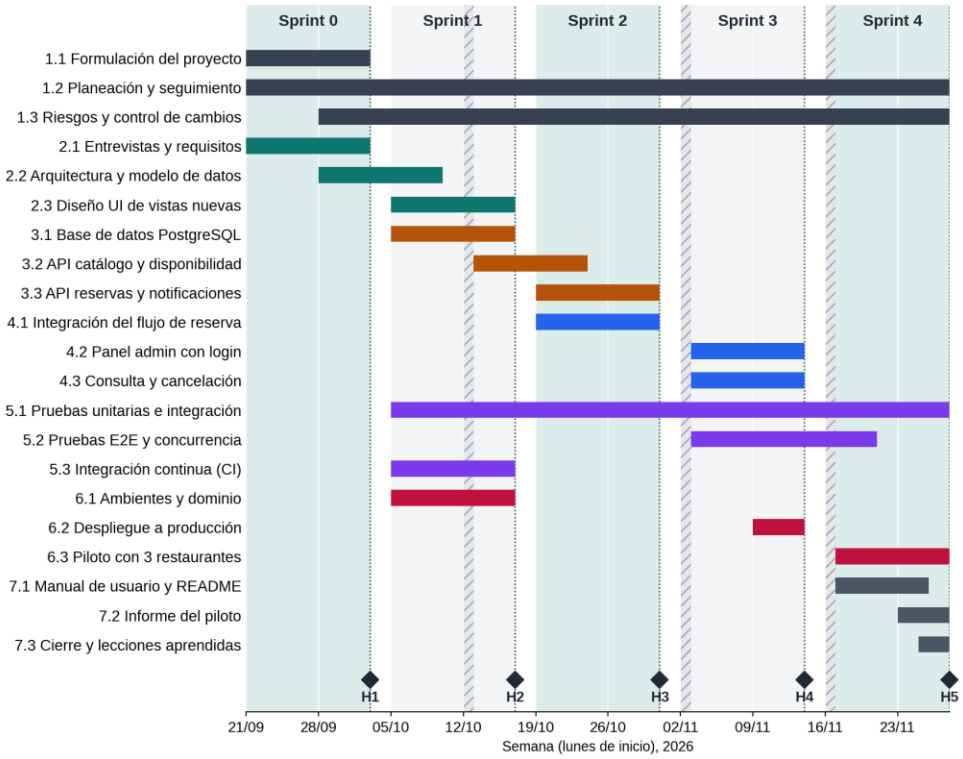
Código	Paquete de trabajo	Inicio	Fin	Días hábiles	Depende de	Responsable
1.1	Formulación del proyecto	21/09	02/10	10	—	Equipo
1.2	Planeación y seguimiento	21/09	27/11	47	—	Kevin
1.3	Riesgos y control de cambios	28/09	27/11	42	1.2	Kevin
2.1	Entrevistas y requisitos	21/09	02/10	10	—	Saleth
2.2*	Arquitectura y modelo de datos	28/09	09/10	10	2.1	José Leonardo
2.3	Diseño UI de vistas nuevas	05/10	16/10	9	2.1	Saleth
3.1*	Base de datos PostgreSQL	05/10	16/10	9	2.2	José Leonardo
3.2	API de catálogo y disponibilidad	13/10	23/10	9	3.1	José Leonardo

Código	Paquete de trabajo	Inicio	Fin	Días hábiles	Depende de	Responsable
3.3*	API de reservas y notificaciones	19/10	30/10	10	3.1, 3.2	José Leonardo
4.1*	Integración del flujo de reserva	19/10	30/10	10	2.3, 3.3	Kevin
4.2*	Panel admin con login	03/11	13/11	9	4.1	Kevin
4.3	Consulta y cancelación	03/11	13/11	9	2.3, 3.3	Saleth
5.1	Pruebas unitarias e integración	05/10	27/11	37	3.1	Equipo
5.2	Pruebas E2E y concurrencia	03/11	20/11	13	4.1	Saleth
5.3	Integración continua (CI)	05/10	16/10	9	2.2	José Leonardo
6.1	Ambientes y dominio	05/10	16/10	9	2.2	José Leonardo
6.2*	Despliegue a producción	09/11	13/11	5	4.2, 6.1	José Leonardo
6.3*	Piloto con 3 restaurantes	17/11	27/11	9	6.2	Saleth
7.1	Manual de usuario y README	17/11	25/11	7	4.2	Kevin
7.2	Informe del piloto	23/11	27/11	5	6.3	Saleth
7.3	Cierre y lecciones aprendidas	25/11	27/11	3	7.2	Equipo

Nota. Elaboración propia. Fechas de 2026 en formato día/mes. Los días hábiles descuentan los festivos del 12 de octubre, 2 de noviembre y 16 de noviembre. Los paquetes 1.2, 1.3 y 5.1 son continuos y acompañan todo el proyecto. * Paquete de la ruta crítica.

Figura 4

Figura 4 Diagrama de Gantt del proyecto



Nota. Elaboración propia. Las franjas rayadas son festivos y los rombos, hitos. Los colores agrupan los paquetes por entregable de la EDT.

La ruta crítica pasa por el modelo de datos (2.2), la base de datos (3.1), la API de reservas (3.3), la integración del flujo (4.1), el panel administrativo (4.2), el despliegue (6.2) y el piloto (6.3): un retraso en cualquiera de ellos corre la fecha de cierre. Las pruebas unitarias (5.1) avanzan en paralelo durante todo el proyecto, en lugar de dejarse para el final.

Para el seguimiento, el Scrum Master mantiene el tablero al día y, al cierre de cada sprint, compara lo terminado con lo planeado en la Tabla 14. Si un paquete de la ruta crítica se atrasa más de dos días hábiles, primero movemos personas desde paquetes que no son críticos y, si eso no alcanza, usamos la holgura prevista en el sprint 4. Si aun así no se recupera la fecha, el ajuste se registra como un cambio en el paquete 1.3 y se informa en la siguiente revisión de sprint, para que nadie se entere del retraso el día de la entrega.

8. Presupuesto

El presupuesto valora todos los recursos del proyecto, aunque varios no impliquen un pago en efectivo. El talento humano se calculó con el salario promedio de un desarrollador junior en Colombia, $2.757.964 mensuales (Indeed, 2026), dividido entre 182 horas al mes, que corresponden a la jornada de 42 horas semanales vigente desde julio de 2026 (Alva, 2025). El resultado es una tarifa de $15.154 por hora. Los servicios en dólares se convirtieron con la tasa representativa del mercado del 20 de octubre de 2026, 3.148,02 pesos por dólar (Banco de la república, 2026).

Tabla 15

Tabla 15 Presupuesto del proyecto en pesos colombianos

Rubro	Concepto	Detalle o supuesto	Cantidad	Valor unitario	Total
Talento humano	Saleth Tatiana Ibarra Martínez (Product Owner, UX y pruebas)	12 h/semana × 10 semanas	120 h	$15.154	$1.818.480
	Kevin Smith Correal Chocontá (Scrum Master y frontend)	12 h/semana × 10 semanas	120 h	$15.154	$1.818.480
	José Leonardo Guzmán Tobón (Líder técnico, backend y despliegue)	12 h/semana × 10 semanas	120 h	$15.154	$1.818.480
Equipos	Depreciación de 3 computadores portátiles propios	Valor de referencia $3.000.000 c/u, depreciación 20 % anual ($50.000/mes c/u)	3 × 2,3 meses	$50.000	$345.000
Software y nube	Hosting frontend (Netlify, plan gratuito)	Ya en uso por el prototipo	—	$0	$0
	Backend en Render: Free en desarrollo	Plan Free en desarrollo (Render, s.f.-b); Starter USD 7/mes en el piloto (Render, s.f.-a); TRM 3.306,86	1 mes	$23.148	$23.148
	Base de datos PostgreSQL (Neon, plan gratuito)	0,5 GB por proyecto, sin vencimiento (Neon, s.f.)	—	$0	$0
	Envío de correos (Resend, plan gratuito)	3.000 correos/mes, 100 por día (Resend, s.f.)	—	$0	$0
	Repositorio, tablero y CI (GitHub, plan gratuito)	GitHub Projects y GitHub Actions	—	$0	$0
	Dominio reservaya.com.co	Registro anual, IVA incluido (MI.COM.CO, s.f.)	1 año	$114.990	$114.990
Operación del piloto	Guía rápida impresa y código QR de mesa	Material para 3 restaurantes	3 kits	$20.000	$60.000
	Subtotal				**$5.998.578**
	Imprevistos (10 %)	Reserva para riesgo R05			$217.952
Total del proyecto					**$6.204.530**

Nota. Elaboración propia. Los valores de depreciación y material son supuestos del equipo. Precios consultados en septiembre de 2026. Los valores correspondientes a equipos y trabajo del equipo representan costos estimados o aportes en especie.

La Tabla 16 resume el presupuesto por rubro y separa lo que el equipo aporta en especie (su tiempo y sus equipos) de lo que requiere un desembolso real.

Tabla 16

Tabla 16 Resumen del presupuesto por rubro

Rubro	Valor	Participación	Tipo de aporte
Talento humano	$5.455.440	90,9 %	En especie (tiempo del equipo)
Equipos	$345.000	5,8 %	En especie
Software y nube	$138.138	2,3 %	Desembolso
Operación del piloto	$60.000	1,0 %	Desembolso
Subtotal	**$5.998.578**	**100 %**	

El talento humano representa casi todo el costo, algo esperable en un proyecto de software. El dinero que el equipo realmente debe desembolsar es $198.138, o $217.952 con el 10% de imprevistos, porque la infraestructura se apoya en planes gratuitos y el único servidor de pago se activa durante el piloto. Si un restaurante adoptara el producto después del semestre, el costo mensual de operación sería cercano a $32.700: $23.148 del servidor y unos $9.580 del dominio prorrateado.

9. Repositorio del proyecto

El código fuente, la documentación y este documento se gestionan en GitHub. La Tabla 17 reúne la información del repositorio y las prácticas de trabajo que el equipo adopta para esta etapa.

Tabla 17

Tabla 17 Información del repositorio

Elemento	Detalle
Repositorio del despliegue	https://github.com/leonardeco/ReservaYa
Repositorio principal del grupo	https://github.com/KevinnC18/ReservaYa (el anterior es una bifurcación)
Despliegue actual del prototipo	https://reservayaok.netlify.app

Elemento	Detalle
Estado actual	Frontend Angular 21 con Tailwind CSS 4, 6 vistas con carga diferida y estado compartido con signals, las 35 pruebas unitarias con Vitest pasan
Ubicación de este documento	Carpeta docs/ del repositorio, en PDF
Estrategia de ramas	main protegida (solo producción); develop para integración; ramas feature/ por historia de usuario, fusionadas con pull request revisado
Tablero del proyecto	GitHub Projects con columnas Backlog, Por hacer, En progreso, En revisión y Hecho
Integración continua	GitHub Actions ejecuta compilación y pruebas en cada pull request (paquete 5.3)

Conclusiones

El desarrollo de la versión 1.0 de ReservaYa se puede considerar técnicamente viable y responde a una necesidad real del sector gastronómico en Colombia, el cual atraviesa dificultades económicas que exigen optimizar cada proceso operativo. Mediante la integración del frontend existente en Angular con una nueva API REST en FastAPI y una base de datos PostgreSQL, se logrará superar la limitación de los datos simulados, garantizando la persistencia de la información y un control riguroso de la disponibilidad por mesa.

La implementación de una arquitectura **cliente-servidor** con restricciones de unicidad y transaccionalidad resuelve los principales puntos críticos de los restaurantes pequeños y medianos como la doble asignación de mesas y el ausentismo sin control (no-show), centralizando la operación en una única fuente de verdad automatizada las 24 horas.

Finalmente, la planeación a través de Scrum, dividida en cinco sprints, permite priorizar los requerimientos esenciales bajo la filosofía de manufactura esbelta. Asimismo, el apalancamiento en herramientas y servicios en la nube de capa gratuita reduce considerablemente el costo real de desembolso a un monto mínimo, haciendo que la plataforma sea económicamente sostenible para pequeños negocios locales.

Referencias

Alva, F. (2025, 19 de diciembre). *Salario mínimo en Colombia 2026*. Alegra. https://blog.alegra.com/colombia/salario-minimo-en-colombia-2026/

Banco de la república. (2026). *Banco de la República*. Obtenido de TRM - Tasa de cambio peso colombiano por dólar: https://suameca.banrep.gov.co/estadisticas-economicas/informacionSerie/1/tasa_cambio_peso_colombiano_trm_dolar_usd

Indeed. (2026). *Sueldo de desarrollador/a junior en Colombia*. https://co.indeed.com/career/desarrollador-junior/salaries

La casa del camarero. (2026, 6 de enero). *¿Por qué es tan importante una buena gestión de reservas?* https://lacasadelcamarero.com/blog/gestion-reservas-evitar-overbooking-restaurantes/

MI.COM.CO. (s.f.). *Precios de dominios*. https://mi.com.co/precios

Múnera Zambrano, M. (2026, 23 de enero). *En un año, más de dos mil restaurantes cerraron en Colombia, dice Acodrés: en "jaque" está el sector gastronómico que pide salvarse al Gobierno.* El Tiempo. https://www.eltiempo.com/colombia/santander/en-un-ano-mas-de-dos-mil-restaurantes-cerraron-en-colombia-dice-acodres-en-jaque-esta-el-sector-gastronomico-que-pide-salvarse-al-gobierno-3526642

Neon. (s.f.). *Pricing*. https://neon.com/pricing

Plaza, A. (2026, 2 de marzo). *Los restaurantes plantan cara al "no-show": más de un 20 % ya pide datos bancarios para evitar las reservas fantasma*. Infobae. https://www.infobae.com/espana/2026/03/02/los-restaurantes-plantan-cara-al-no-show-mas-de-un-20-ya-pide-datos-bancarios-para-evitar-las-reservas-fantasma/

Pressman, R. S. (2021). *Ingeniería de software* (9.ª ed.). McGraw-Hill Interamericana.

Project Management Institute. (2021). *La guía de los fundamentos para la dirección de proyectos (Guía del PMBOK) y el estándar para la dirección de proyectos* (7.ª ed.).

Render. (s.f.-a). *Pricing*. https://render.com/pricing

Render. (s.f.-b). *Deploy for free*. https://render.com/docs/free

Resend. (s.f.). *Pricing*. Recuperado el 28 de septiembre de 2026, de https://resend.com/pricing

Sommerville, I. (2005). *Ingeniería del software* (7.ª ed.). Pearson Educación.